



Core Java

Exception

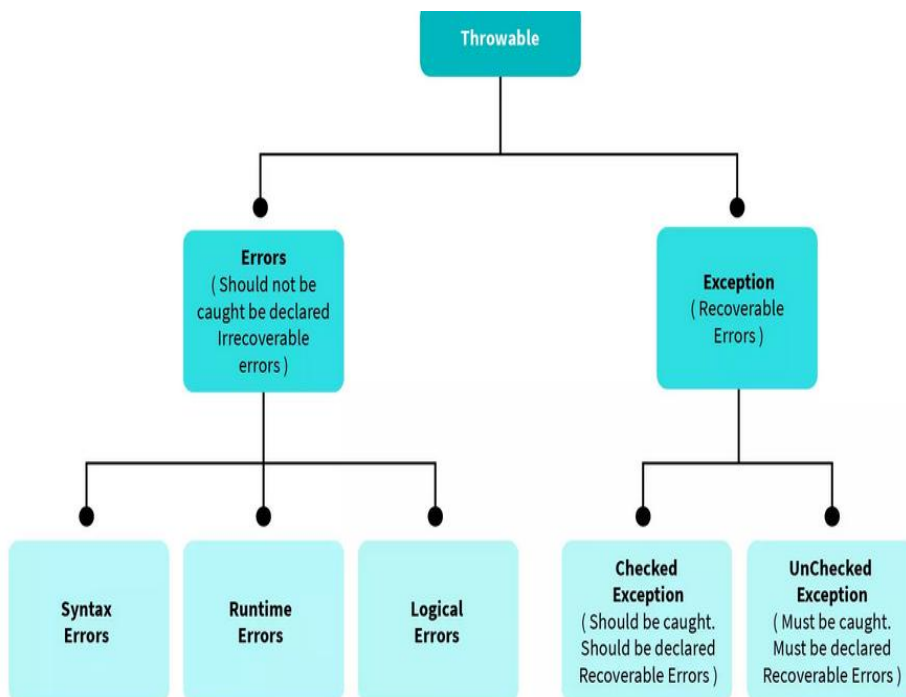
Outline

- Classification of Exceptions
- Exceptions and Error Classes
- Checked and Unchecked Exception
- Throw an Exception
- Exception Handling Techniques, Try-Catch and Finally Clause,
- User Defined Exception



Errors to Exceptions

- In Java, both Error and Exception are subclasses of Throwable, with Error often indicating severe system resource issues, while Exceptions can occur during runtime and disrupt normal program flow, often being manageable through specific code constructs.
- Errors are system-level issues that cannot typically be recovered from, while Exceptions are application-level anomalies that can be caught and handled by the developer.



Exception

- An **exception** is an object that describes an **unusual** or **erroneous situation**.
- Exceptions are thrown by a program, and may be caught and handled by another part of the program.
- A program can be separated into a **normal execution flow** and an **exception execution flow**.
- An error is also represented as an object in Java, but usually represents a unrecoverable situation and should not be caught.
- Java has a predefined set of exceptions and errors that can occur during execution.
- A program can deal with an exception in one of three ways:
 - **ignore** it
 - **handle** it where it occurs
 - handle it at **another place** in the program

Exception

- The manner in which an exception is processed is an important design consideration.
- **How exceptions handled in Java?**
 - In java, the exception handling mechanism uses five keywords namely **try**, **catch**, **finally**, **throw**, and **throws**.



key differences between errors and exceptions in Java:

Aspect	Error	Exception
Definition	Serious problems indicating a failure in the runtime environment.	Issues that occur during program execution and can be handled.
Recoverability	Generally not recoverable by the application.	Can be handled using try-catch blocks.
Examples	OutOfMemoryError, StackOverflowError, NoClassDefFoundError.	IOException, ArithmeticException, NullPointerException.
Hierarchy	Derived from the Error class in the java.lang package.	Derived from the Exception class in the java.lang package.
When Occurs	Indicates problems with the system or environment.	Occurs due to coding errors or input/output problems.

key differences between errors and exceptions in Java:

Aspect	Error	Exception
Impact	Causes application termination, often irrecoverable.	Application can continue after handling the exception.
Handling Mechanism	Cannot be handled using try-catch.	Can be handled using try-catch blocks or throws declaration.
Use Case	For critical system-level issues (e.g., memory errors).	For logical issues or unexpected situations in the application.
Custom Subclasses	Creating custom subclasses is uncommon and not recommended.	Developers can create custom exceptions by extending the Exception class.

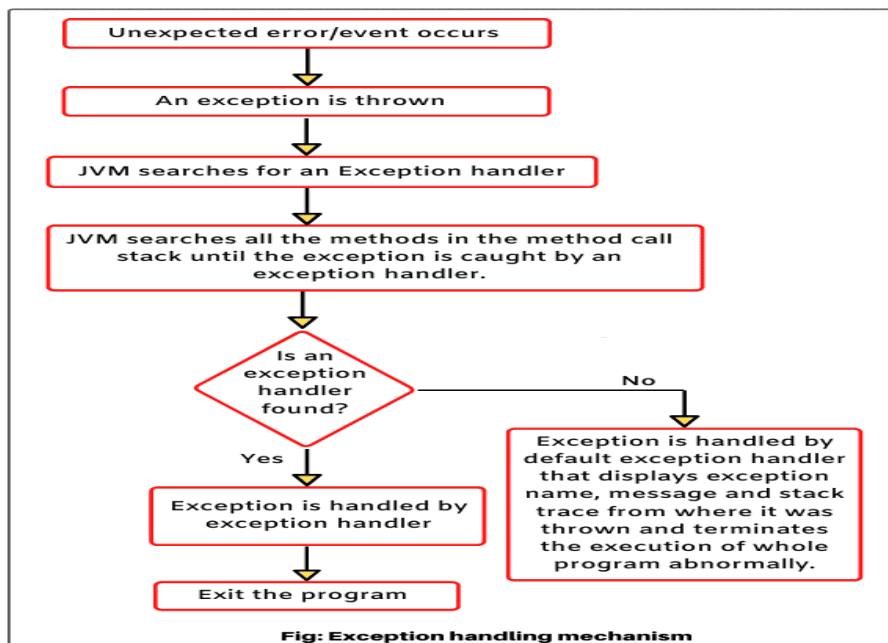
Flow of an exception.



Exception Handling Keywords



How Does Exception Handling Mechanism Work?



Exception handling key words

- **throw** : The Java runtime creates an exception object as soon as an error occurs and begins processing the exception. In some cases, we may need to generate exceptions. Null passwords, for example, should be thrown by an authentication program to clients. The **throw** keyword allows you to throw exceptions to the runtime.
- **throws** : Methods that throw an exception and don't handle it must use the throws keyword in their signature to inform the caller program of the exceptions they might throw. With the **throws** keyword, a caller method can handle these exceptions or propagate them to one of its caller methods. With the throws clause, we can specify multiple exceptions, including the main() method.

Exception handling key words

- **try-catch** : For handling exceptions, we use a try-catch block. To handle exceptions, the try block starts with the keyword **try** and ends with the **catch** block. You can use more than one catch block for one try block. Try-catch blocks can also be nested. You need to specify an Exception type as a parameter in the catch block.
- **finally** : In a try-catch block, the **finally** block is optional. When an exception occurs, the execution process gets stopped, so it might be necessary to use the final block if we have some resources open that won't be closed when the exception occurs. Regardless of whether an exception occurs, the final block always executes.

Using try and catch

- The keyword **try** is used to define a block of code that will be tests the occurrence of an exception.
- The keyword **catch** is used to define a block of code that handles the exception occurred in the respective try block.
- Example:

```
try{
    // code that may cause exception
}
catch(Exception e){
    // code when exception occurred
}
```

Exception is the superclass of all the exception that may occur in Java

Using try and catch

- Multiple catch:

```
try{
    // code that may cause exception
}
catch(ArithmeticException ae){
    // code when arithmetic exception occurred
}
catch(ArrayIndexOutOfBoundsException aiobe){
    // when array index out of bound exception occurred
}
```

Nested try statements

```
try
{
    try
    {
        // code that may cause array index out of bound exception
    }
    catch(ArrayIndexOutOfBoundsException aiobe)
    {
        // code when array index out of bound exception occurred
    }
    // other code that may cause arithmetic exception
}
catch(ArithmeticException ae)
{
    // code when arithmetic exception occurred
}
```

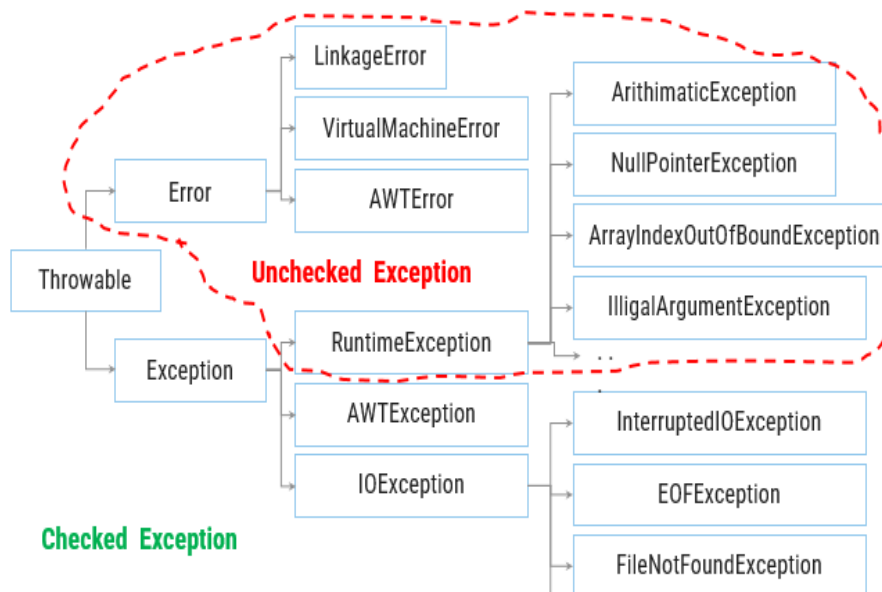
Types of Exceptions

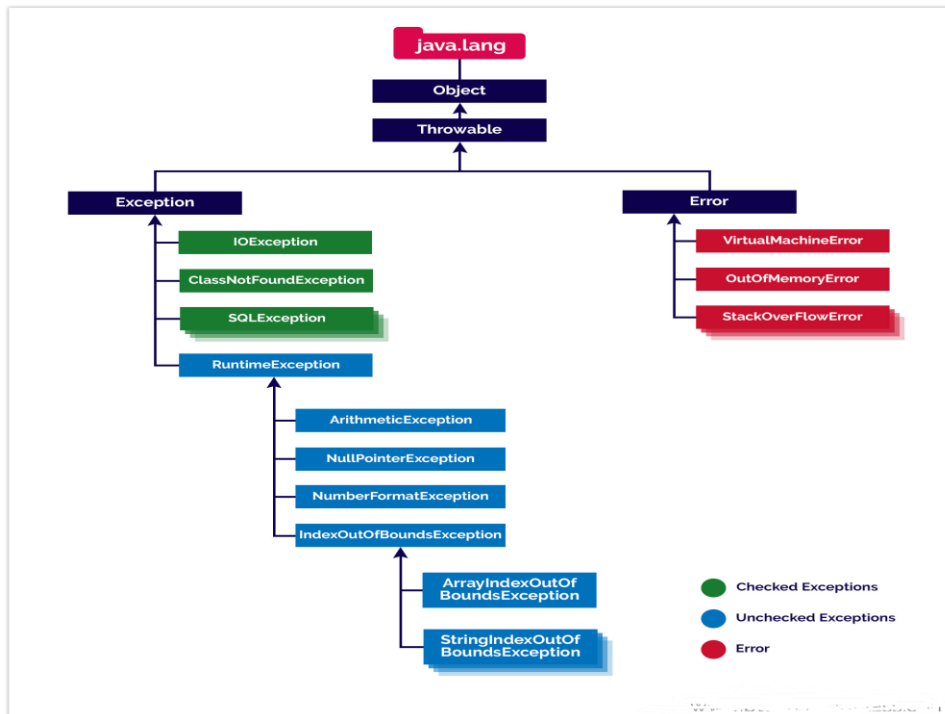
- An exception is either checked or unchecked.
- **Checked** Exceptions : A checked exception either must be caught by a method, or must be listed in the throws clause of any method that may throw or propagate it.
 - The compiler will issue an error if a checked exception is not caught or asserted in a throws clause
 - Example: IOException, SQLException etc...
- **Unchecked** Exceptions : An unchecked exception does not require explicit handling, though it could be processed using try catch.
 - The only unchecked exceptions in Java are objects of type RuntimeException or any of its descendants.
 - Example: ArithmeticException, ArrayIndexOutOfBoundsException, NullPointerException etc...

The Exception Class Hierarchy

- Classes that define exceptions are related by inheritance, forming an exception class hierarchy.
- All error and exception classes are descendants of the Throwable class
- The custom exception can be created by extending the Exception class or one of its descendants.

The Exception Class Hierarchy





Java's Inbuilt Unchecked Exceptions

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ClassCastException	Invalid cast.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.

Unchecked Exceptions example

- In the exception class hierarchy, the unchecked exception classes are the children of RuntimeException class, which is a child class of Exception class.
- The unchecked exception is also known as a runtime exception.

Java's Inbuilt checked Exceptions

Exception	Meaning
ClassNotFoundException	Class not found.
IOException	Input Output Exceptions
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.

Checked Exceptions example

- In the exception class hierarchy, the checked exception classes are the direct children of the Exception class.
- The checked exception is also known as a compile-time exception.

Uncaught Exceptions in Java example

- The Division by zero exception is one of the example for uncaught exceptions. Look at the following code.

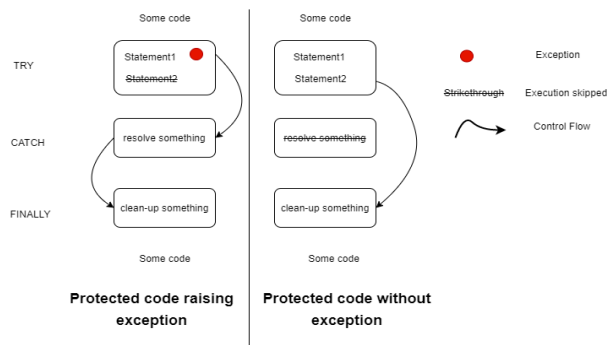
throw statement

- It is possible for your program to throw an exception **explicitly**, using the **throw** statement.
- The general form of throw is shown here:

```
throw ThrowableInstance;
```
- Here, **ThrowableInstance** must be an object of type **Throwable** or a **subclass** of **Throwable**.
- The throw keyword must be used inside the try block.
- Primitive types, such as int or char, as well as non-throwable classes, such as String and Object, cannot be used as exceptions.
- There are two ways you can obtain a Throwable object:
 - using a parameter in a catch clause,
 - or creating one with the new operator.

final statement

- The purpose of the **finally** statement will allow the execution of a segment of code regardless if the try statement throws an exception or executes successfully.
- The advantage of the **finally** statement is the ability to clean up and release resources that are utilized in the **try** segment of code that might not be released in cases where an exception has occurred.



throws statement

- A throws statement lists the types of exceptions that a **method** might throw.
- This is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the throws clause. If they are not, a compile-time error will result.
- This is the general form of a method declaration that includes a **throws clause**:

```
type method-name(parameter-list) throws exception-list {
    // body of method
}
```
- Here, *exception-list* is a comma-separated list of the exceptions that a method can throw.

throws statement

- The throws keyword specifies the exceptions that a method can throw to the default handler and does not handle itself.
- That means when we need a method to throw an exception automatically, we use throws keyword followed by method declaration
- When a method throws an exception, we must put the calling statement of method in try-catch block.
- Example :

```
void myMethod() throws ArithmeticException,
    NullPointerException
{
    // code that may cause exception
}
```

Create Your Own Exception

- Although Java's built-in exceptions handle most common errors, you will probably want to create your own exception types to handle situations specific to your applications.
- This is quite easy to do: just define a subclass of `Exception` (which is, of course, a subclass of `Throwable`).
- The `Exception` class does not define any methods of its own. It does inherit those methods provided by `Throwable`.
- Thus, all exceptions have methods that you create and defined by `Throwable`.
- The user must override the `toString()` method in the user-defined exception class.
- We may create constructor in the user-defined exception class and pass a string to `Exception` class constructor using `super()`. We can use `getMessage()` method to access the string.

Methods of Exception class

Method	Description
<code>Throwable fillInStackTrace()</code>	Returns a <code>Throwable</code> object that contains a completed stack trace. This object can be rethrown.
<code>Throwable getCause()</code>	Returns the exception that underlies the current exception. If there is no underlying exception, null is returned.
<code>String getMessage()</code>	Returns a description of the exception.
<code>StackTraceElement[] getStackTrace()</code>	Returns an array that contains the stack trace, one element at a time, as an array of <code>StackTraceElement</code> .
<code>Throwable initCause(Throwable causeExc)</code>	Associates <code>causeExc</code> with the invoking exception as a cause of the invoking exception. Returns a reference to the exception.
<code>void printStackTrace()</code>	Displays the stack trace.
<code>void printStackTrace(PrintStream stream)</code>	Sends the stack trace to the specified stream.
<code>void setStackTrace(StackTraceElement elements[])</code>	Sets the stack trace to the elements passed in elements.
<code>String toString()</code>	Returns a <code>String</code> object containing a description of the exception.

final vs finally vs finalize

final	finally	finalize
It is a keyword.	It is a block.	It is a method.
Used to apply restrictions on class, methods & variables.	Used to place an important code.	Used to perform clean-up processing just before the object is garbage collected.
final class can't be inherited, method can't be overridden & the variable value can't be changed.	It will be executed whether the exception is handled or not.	

throw vs throws

throw	throws
1. Used to explicitly throw an exception	1. Used to declare an exception
2. Checked exceptions cannot be propagated using throw only	2. Checked exceptions can be propagated
3. Followed by an instance	3. Followed by a class
4. Used within a method	4. Used with a method signature
5. Cannot throw multiple exceptions	5. Can declare multiple exceptions

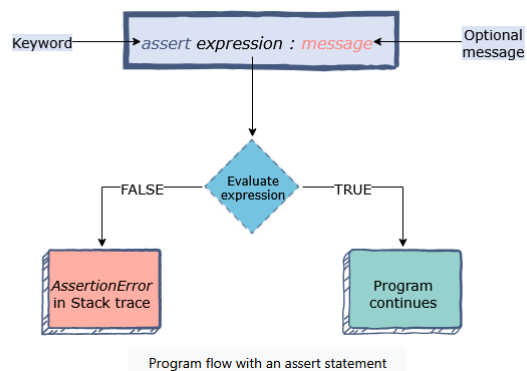


Core Java

Assertion

Outline

- Assertion Concept
- Enable Assertion, Disable Assertion.



Assertion

- An assertion is defined as a claim or assumption.
- In programming, there are cases when we, as programmers, know that a certain variable takes a specific value(s) at a particular line of code. We usually represent such a claim using comments in our code.
- An assertion in Java is one single line of code which checks a certain boolean expression and JVM throws a ***AssertionError*** if that expression is *false*. We say that an assertion has *failed* in this case.
- **Note:** Assertions are disabled by default in Java. Include -ea command in your Java interpreter (or search specifically for your IDE on the web).

Syntax

- There are two ways to write an assertion using the assert keyword
`assert expression;`
- We can pass our own message as any type of variable (other than void) which will be shown in the stack trace in case the expression was false.
`assert expression : "Invalid value";`

Why use Java Assertion

- Wherever a software engineer needs to check whether his/her suspicions are right or not,
 - To ensure that an inaccessible code is really inaccessible.
 - Ensure that presumptions written in the comments are correct.
 - To ensure that the default switch case is not reached.
 - Check the state of the object.
 - At the start of the method.
 - After the invocation of the method.
- JUnit itself is the biggest manifestation of what assertion in Java can do. Using assertion along with Exception you can write robust code.

When to use assertions?

- Using an assertion instead of a comment we put for our own convenience:

Comment	Assertion
<pre> 1 int x = 3; 2 3 if(x <= 2) { 4 // ... 5 } 6 else { // x > 2 7 // ... 8 }</pre>	<pre> 1 int x = 3; 2 3 if(x <= 2) { 4 // ... 5 } 6 else { 7 assert x > 2 : "x is not greater 2"; 8 }</pre>

When to use assertions?

- Checking control-flow. If we know that our code is not meant to reach a certain line, we can place the assertion in that prohibited area; but, it should be written in such a way that it always returns false:

Normal	<pre> 1 int i = 5; 2 while(i > 0){ 3 if(i == 2){ // Can be any condition which you think will be surely met. 4 return; 5 } 6 7 i--; 8 } 9 // My program should not reach this place. </pre>
Assertion	<pre> 1 int i = 5; 2 while(i > 0){ 3 if(i == 2){ // Can be any condition which you think will be surely met. 4 return; 5 } 6 7 i--; 8 } 9 assert false : "Condition not met."; </pre>

Java Assertion vs Normal Exception Handling

- We mainly use Assertions to check logically impossible situations. For example, we can use them to check the state a code expects before it starts running or state after it finishes running.
- The difference between assertions and normal exception handling is that the assertions are generally disabled at run-time.

Where Not to Utilize Assertion in Java

- There are many situations where we should not attempt to use Java Assertions:
 - To replace the error messages.
 - For checking arguments in the public methods as they may be provided by the user.
 - To handle errors provided by the user, for this we must use error handling
 - lastly on command line arguments.

Handling an AssertionError

- The class `AssertionError` extends `Error` class and the `Error` class itself extends `Throwable`. This means that the `AssertionError` is an unchecked exception.
- Therefore we do not need to declare the methods that use assertions and also further calling code needs not try and catch them. We use `AssertionErrors` to indicate unrecoverable conditions in an application, so we should never try to handle them or attempt recovery.

Enabling and Disabling Assertions in Java

- We can enable Java Assert by default. The syntax for the same is:

java -ea Test

Or

Java -enableassertions Test

- The syntax for disabling Java Assert-

java -da Test

Or

java -disableassertions Test

Assertions Example [to run]

- Open the **Run and Debug** view in VS Code (Ctrl+Shift+D).
- If you don't have a launch.json file, VS Code will prompt you to create one when you run your code. Select the appropriate Java environment.
- Once the launch.json file is open, locate the configuration for your main class (e.g., "AssertionExample").
- Add a "vmArgs" property with the value "-enableassertions" (or "-ea") to that specific configuration.

Assertion vs Normal Exception Handling

Aspect	Assertion	Exception Handling
Purpose	Check logically impossible states	Handle runtime errors
Runtime Behavior	Can be disabled	Always executed
Typical Use	Development/testing	Production
Example	Precondition/Postcondition checks	User input validation, IO errors

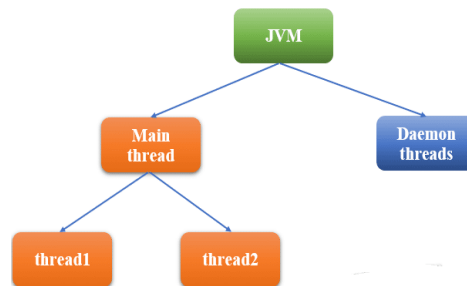
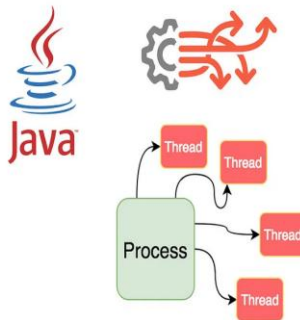


Core Java

Threads

Outline

- Introduction
- Thread States
- Thread Properties (includes Virtual Thread)
- Implementation of Thread Class and Runnable Interface
- Thread Synchronization.



Thread

- **Process:** The execution of the program is referred to as the process. On another hand, we can say when we are executing a program it means a process is in running. A program can be divided into small processes(subprocesses) also.
- **Thread:** A thread is the smallest unit of process or a lightweight sub-process.
 - A program can have any number of sub-processes and each sub-process can run parallely by a thread.
 - A thread behaves like a separate CPU executing the process, but the CPU shares the time between threads.
 - Each program has at least one thread. If we create a simple program to print “Hello world”, it means at least one thread will exist in the program.

What is Multithreading?

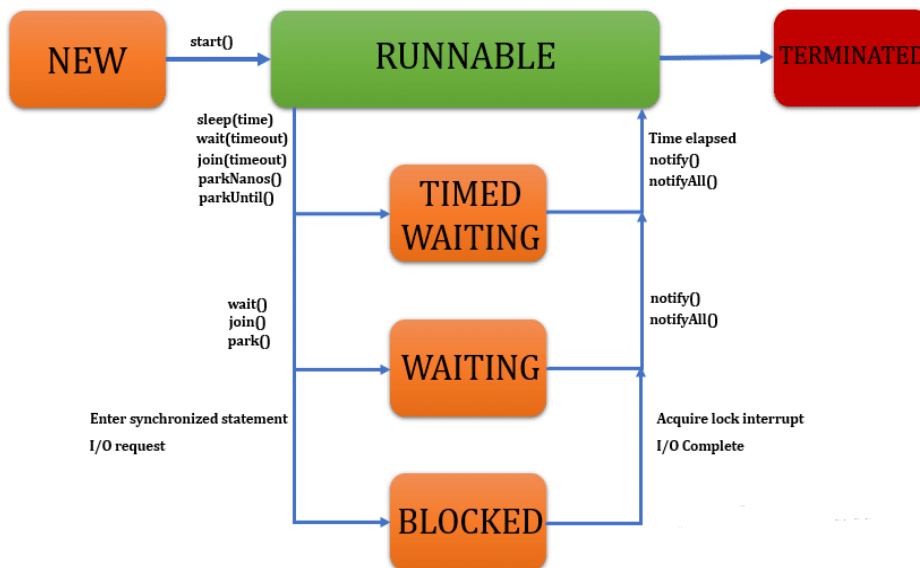
- Multithreading in Java is a process of *executing multiple threads* simultaneously.
- A thread is a *lightweight sub-process*, the smallest unit of processing.
- Multiprocessing and multithreading, both are used to achieve multitasking.
- Threads use a *shared memory area*. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.
- A thread goes through various stages in its life cycle. For example, a thread is born, started, runs, and then dies.



Important points about multithreading in Java

- Multithreading is used to utilize the maximum time of the CPU. In multithreading, we can execute two or more parts of a program simultaneously. Each part of the program can run separately by single or multiple threads.
- A Thread is a lightweight sub-process, that can separate from other threads. The multiple threads can share a common memory space. The CPU divides the time for all thread and switch from one thread to another thread for a while.
- A thread runs independently because it has a separate path of execution.

Life cycle of a Thread

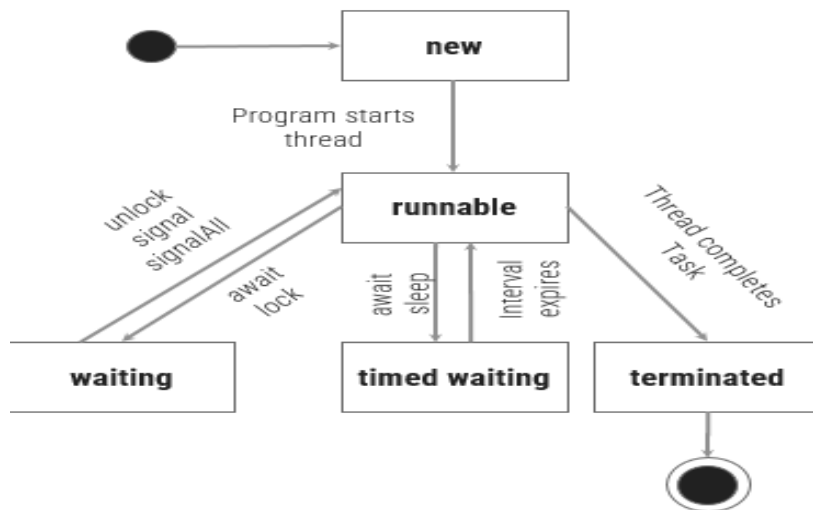


Life cycle of a Thread [Stages]

- **New:** A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a born thread.
- **Runnable:** After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
- **Waiting:** Sometimes a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals waiting thread to continue.
- **Timed waiting:** A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.

Life cycle of a Thread [Stages]

- **Terminated:** A runnable thread enters the terminated state when it completes its task or otherwise terminates.



Properties of thread

- **Lightweight process:** A thread is a lightweight process in a program. A program can have any number of sub-processes that are handled by threads. That's why we call the thread a lightweight process.
- **Sharing resources:** Multiple threads can share resources like data and code. A thread can make a change in data that are used by another thread. Even this concept leads to the duplicity of data. We will discuss it later.
- **At least one thread:** Each program has at least one thread. Whenever we create a program the JVM automatically creates a thread. A program can contain any number of threads.

Properties of thread

- **Thread priority:** Each thread has a priority that lies between 1 to 10. The JVM automatically assigns the priority to each thread but we can do it by **setPriority() method**. The default priority of a thread is 5. The JVM executes the higher priority thread before the lower priority. If two thread has the same priority the thread scheduler decides which thread run first.
- **Creation of thread:** A thread can create in two ways either by **Thread class** or **Runnable interface**.
We can create a thread by extending the **Thread class** or by implements the **Runnable interface**. The Thread class provides some methods and constructors that can be used to perform operations.

Properties of thread

- **Types of the thread:** In java, we can create types of thread: **user threads** or daemon threads. The **user thread** has a higher priority than the **daemon thread**. A **daemon thread** always works in the background and JVM can ignore them before terminating the process. But for the user thread, the JVM could not terminate the execution before the completion of the user thread. Memory management, system management, and signal processing are managed by daemon threads
- **Life cycle of thread :** A thread goes in different states during the execution of the program. It has 6 states those are:
 - New state, Runnable state, Running state(This state doesn't exist), Blocked state, Waiting state, Timed waiting, Terminated state .

Properties of thread

- **Thread scheduler:** The execution of the thread totally depends on the **thread scheduler**. A **thread scheduler** decides which thread should be run first. The **thread scheduler** depends upon the JVM. The JVM can indicate to the **thread scheduler** those threads are more important or critical.
- **Stack:** Each thread stores the local variables and method call in its own stack. The size of this stack depends on the JVM and it can be controlled using -XX:ThreadStackSize JVM option

Important Points about java thread sleep

- The **sleep() method** always pauses the current thread execution. When the JVM finds the sleep() method in the code, it checks which thread is running and pause the execution of the thread. After the pause of the current thread, the thread scheduler assigns the CPU to another thread if any thread exists.
- When we use the **sleep() method** to pause the execution of the thread. The **thread scheduler** schedule other thread for CPU time. So, there is no guarantee that the thread wakes up exactly after the time specified in the **sleep() method**. It totally depends on the **thread scheduler**.
- When any thread is sleeping and any interruption by another thread then it throws **InterruptedException**.
- Whenever we want to use the **sleep() method** we also need to handle the **InterruptedException**. If we will not handle it, the JVM will show a compilation error.

Constructors of Thread Class

Constructor	Action Performed
Thread()	Allocates a new Thread object.
Thread(Runnable target)	Allocates a new Thread object.
Thread(Runnable target, String name)	Allocates a new Thread object.
Thread(String name)	Allocates a new Thread object.
Thread(ThreadGroup group, Runnable target)	Allocates a new Thread object.

Constructors of Thread Class

Constructor	Action Performed
Thread(ThreadGroup group, Runnable target, String name)	Allocates a new Thread object so that it has targeted as its run object, has the specified name as its name, and belongs to the thread group referred to by a group.
Thread(ThreadGroup group, Runnable target, String name, long stackSize)	Allocates a new Thread object so that it has targeted as its run object, has the specified name as its name, and belongs to the thread group referred to by group, and has the specified stack size.
Thread(ThreadGroup group, String name)	Allocates a new Thread object.

Methods of Thread Class

Methods	Action Performed
activeCount()	Returns an estimate of the number of active threads in the current thread's thread group and its subgroups
checkAccess()	Determines if the currently running thread has permission to modify this thread
clone()	Throws CloneNotSupportedException as a Thread can not be meaningfully cloned
currentThread()	Returns a reference to the currently executing thread object

Methods of Thread Class

Methods	Action Performed
<code>dumpStack()</code>	Prints a stack trace of the current thread to the standard error stream
<code>enumerate(Thread[] tarray)</code>	Copies into the specified array every active thread in the current thread's thread group and its subgroups
<code>getAllStackTraces()</code>	Returns a map of stack traces for all live threads
<code>getContextClassLoader()</code>	Returns the context ClassLoader for this Thread

Methods of Thread Class

Methods	Action Performed
<code>getDefaultUncaughtExceptionHandler()</code>	Returns the default handler invoked when a thread abruptly terminates due to an uncaught exception
<code>getId()</code>	Returns the identifier of this Thread
<code>getName()</code>	Returns this thread's name
<code>getPriority()</code>	Returns this thread's priority

Methods of Thread Class

Methods	Action Performed
getStackTrace()	Returns an array of stack trace elements representing the stack dump of this thread
getState()	Returns the state of this thread
getThreadGroup()	Returns the thread group to which this thread belongs
getUncaughtExceptionHandler()	Returns the handler invoked when this thread abruptly terminates due to an uncaught exception

Methods of Thread Class

Methods	Action Performed
holdsLock(Object obj)	Returns true if and only if the current thread holds the monitor lock on the specified object
interrupt()	Interrupts this thread
interrupted()	Tests whether the current thread has been interrupted
isAlive()	Tests if this thread is alive

Methods of Thread Class

Methods	Action Performed
isDaemon()	Tests if this thread is a daemon thread
isInterrupted()	Tests whether this thread has been interrupted
join()	Waits for this thread to die
join(long millis)	Waits at most millis milliseconds for this thread to die

Methods of Thread Class

Methods	Action Performed
run()	If this thread was constructed using a separate Runnable run object, then that Runnable object's run method is called; otherwise, this method does nothing and returns
setContextClassLoader(ClassLoader cl)	Sets the context ClassLoader for this Thread
setDaemon(boolean on)	Marks this thread as either a daemon thread or a user thread
setDefaultUncaughtExceptionHandler(Thread.UncaughtExceptionHandler eh)	Set the default handler invoked when a thread abruptly terminates due to an uncaught exception, and no other handler has been defined for that thread

Methods of Thread Class

Methods	Action Performed
setName(String name)	Changes the name of this thread to be equal to the argument name.
setUncaughtExceptionHandler(Thread.UncaughtExceptionHandler eh)	Set the handler invoked when this thread abruptly terminates due to an uncaught exception
setPriority(int newPriority)	Changes the priority of this thread
start()	Causes this thread to begin execution; the Java Virtual Machine calls the run method of this thread

Methods of Thread Class

Methods	Action Performed
sleep(long millis)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds, subject to the precision and accuracy of system timers and schedulers
toString()	Returns a string representation of this thread, including the thread's name, priority, and thread group
yield()	A hint to the scheduler that the current thread is willing to yield its current use of a processor

Creating a Thread in Java

- There are two ways to create a Thread
 - **extending the Thread class**
 - **implementing the Runnable interface**
- But the **Creation of a thread in java** is always dependent upon the situation and the scenarios. In both ways, we provide the body of the thread in the **run()** method and start the execution of the thread by the **start()** method.

Creation of thread by extending thread class

- We can create a thread by extending the **Thread class** that exists in **java.lang** package. As a result, We need to extend the Thread class, in our class, and create an object/instance of that class. We must override the **run()** method that is present in the **Thread class**.
- **Step 1:** Create a class in which you want to create a new thread.
- **Step 2:** Extend the **Thread class** by **extends** the keyword.
- **Step 3:** Override the **run()** method of the **Thread class** and provide the body to **run()** method. Finally, The body of the **run()** method is considered the body of the thread that we want to create.
- **Step 4:** Create the object of the class.
- **Step 5:** Call the **start()** method to run the execution of the thread.

Creation of thread using Runnable interface

- We have two ways to create the thread through the **Runnable interface**. We can create a thread by implements the **Runnable interface** or by use of the **anonymous class**.
- We can create a thread by implementing the **Runnable interface** that exists in **java.lang** package. We need to **implement** the **Runnable interface**, in our class. The **Runnable interface** has only one method which is the **run() method**. We must provide the body with the **run() method**.
- `public interface Runnable{public void run();}`

Creation of thread using Runnable interface

- **Step 1:** Create a class in which you want to create a new thread.
- **Step 2:** Implement the **Runnable interface** by **implements** keywords.
- **Step 3:** Provide the body to the **run() method** of **Runnable**. The body of the **run() method** is considered the body of the thread that we want to create.
- **Step 4:** Create the object of the class that implements the **Runnable interface**.
- **Step 5:** Create an object of the **Thread class** by use of a constructor that takes the object of Runnable types as a parameter. For eg: `new Thread(Runnable target)`
 - Instance of **Thread** class is created using following constructor.

`Thread(Runnable threadOb, String threadName);`

Creation of thread using Runnable interface

- Here `threadOb` is an instance of a class that implements the `Runnable` interface and the name of the new thread is specified by `threadName`.
- **Step 6:** Use the thread object to call the **`start()` method** to run the execution of the thread.

Creation of thread using anonymous class [example]

we use the anonymous class **`new Thread()`** to create a thread by extending the `Thread` class anonymously and overriding its **`run()` method**:

thread sleep()

- The **sleep() method** is used to pause the execution of a thread for a specific time.
- The **sleep() method** is a **static method** and it exists in the Thread class.
- We can use it to sleep a thread for a millisecond or millisecond and nanoseconds.
- It is an overloaded method and there are two flavors of the sleep() method.

public static native void sleep(long millis) throws
InterruptedException;

public static void sleep(long millis, int nanos) throws
InterruptedException

Important Points about java thread sleep

- The **sleep() method** always pauses the current thread execution. When the JVM finds the sleep() method in the code, it checks which thread is running and pause the execution of the thread. After the pause of the current thread, the thread scheduler assigns the CPU to another thread if any thread exists.
- When we use the **sleep() method** to pause the execution of the thread. The **thread scheduler** schedule other thread for CPU time. So, there is no guarantee that the thread wakes up exactly after the time specified in the **sleep() method**. It totally depends on the **thread scheduler**.
- When any thread is sleeping and any interruption by another thread then it throws InterruptedException.
- Whenever we want to use the **sleep() method** we also need to handle the **InterruptedException**. If we will not handle it, the JVM will show a compilation error.

Synchronization in java

- In a **multithreading** environment, multiple threads can try to access the same resources and can produce erroneous and unforeseen results.
- Java provides a way to control the access of multiple threads to any shared resource.
- **Synchronization in java** is a process where we want to allow only one thread to access the shared resource at a time.
- So that multiple threads can work together without creating any problems.
- The synchronization process can achieve by **synchronized keyword in java**.
- The **Synchronization keyword** makes the code thread-safe.

Synchronization in java



- **Types of Synchronization**
 - **Method Synchronization:** Locks the entire method to ensure only one thread executes it at a time.
 - **Block Synchronization:** Locks only the critical section of code within a method.

Implements Runnable Vs Extends Thread In Java

Extends Thread	Implements Runnable
We have to override run method	We must implement run method
Inheritance: If extends thread than we can't extends another class because multiple inheritance not allowed in java	If we implements Runnable interface than we can extends another class.
Loosely-coupled: By extends Thread class our class become tightly coupled.	By implements Runnable interface both Task and Executor become loosely coupled. Thread is separate form the job of Threads.
Our class become overheated by extends thread class due to additional methods in thread class. Means inheriting all the functions of the Thread class which we may do not need	No overhead on any method because Runnable interface has no other methods then run().

Implements Runnable Vs Extends Thread In Java

Extends Thread	Implements Runnable
Maintenance of code not easy.	Maintenance of code is easy.
Functions overhead: Thread class creates a unique object and get associated with that object. So that more memory required.	Thread defined by implementing Runnable interface shares the same object. Multiple threads share the same objects. So that less memory is used.
Reusability : By extends Thread in class, its contains both thread and job specific behavior code. So that once thread completes its execution , it can't be restart again.	By implements Runnable interface, we can create a different Runnable class for a specific behavior of job. It provides the freedom to reuse the specific behavior job when required.

Implements Runnable Vs Extends Thread In Java

	Implements Runnable	Extends Thread
Inheritance option	extends any java class	No
Reusability	Yes	No
Object Oriented Design	Good,allows composition	Bad
Loosely Coupled	Yes	No
Function Overhead	No	Yes

What Are Virtual Threads?

- Virtual threads are lightweight threads that simplify writing and maintaining concurrent applications with high compute requirements.
- Traditional platform threads are very heavy, i.e., they use a significant amount of compute resources, and are bound 1:1 to the OS thread, which means that execution time of the threads is dependent on the CPU's execution time.
- Each of these threads would end up utilizing as much as 10 GB of memory; this adds up quickly depending on any level of complexity implemented in your application. These threads were designated to use a single thread with every request. While traditional threads provide a level of simplicity for developing applications, the lack of flexibility often resulted in large applications being negatively impacted by the amount of requests made by the code.

What Are Virtual Threads?

- Virtual threads, on the other hand, move the responsibility of scheduling threads from the OS to the JVM (Java Virtual Machine), effectively introducing an abstraction layer between the OS and the application.
- Virtual threads were created with a few distinct purposes in mind.
 - First, they enable Java developers to write server applications using a simple thread-per-request style.
 - Second, virtual threads enable Java developers to utilize hardware to the fullest extent possible by eliminating the OS as the limiting factor to performance. [in java 21]

Virtual Thread Dos and Don'ts

- While Java developers are still experimenting in how to best implement virtual threads in their workflows, there are a few best practices to keep in mind.
- Start a new virtual thread for every task.
- Virtual threads should not be pooled.
- Most virtual threads will have shallow call stacks and thus be short-lived.

Virtual Thread

- Using the Thread Class (Simplest Method)
 - You can create and start a virtual thread in a single line using the static **Thread.startVirtualThread()** method:
 - // Example: Creating a simple virtual thread


```
Thread.startVirtualThread(() -> {
    System.out.println("Running in: " +
    Thread.currentThread());});
```
- Using Thread.Builder
 - For configuration options like naming, use `Thread.ofVirtual()`
 - // Example: Creating a named virtual thread


```
Thread.ofVirtual().name("my-vthread").start(() -> {
    System.out.println("Named Thread: " +
    Thread.currentThread());});
```

Virtual Thread

- Using ExecutorService
 - For high-throughput, use `Executors.newVirtualThreadPerTaskExecutor()` to create a new virtual thread for each submitted task
 - ```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

// Example: Using ExecutorService for multiple tasks
try (ExecutorService executor =
 Executors.newVirtualThreadPerTaskExecutor()) {
 executor.submit(() -> System.out.println("Virtual Task"));
}
// Auto-closes and waits for tasks
```

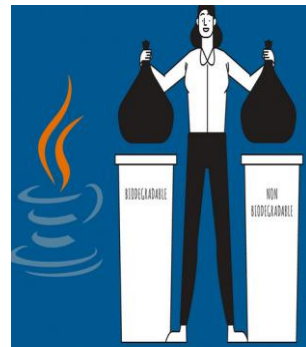


# Core Java

## Garbage Collection

### Outline

- Introduction
- Enhancements to switch
- Text Blocks
- Type inference
- Unnamed Classes and Variables
- Unnamed Patterns and Variables
- String Templates,



## What is Garbage Collection in Java?

- In simpler term **Garbage** means **waste**. But in terms of Java, Garbage means unreferenced objects.
- By means of unreferenced objects that are in heap memory, but they are useless. They are not in use in application anywhere and they should be destroyed.
- The process of removing unused objects from heap memory is known as **Garbage collection**.
- The **garbage collection** process is handled by **Garbage Collector** and it performs an operation for automatic memory management.

## How to Dereference an Object in Java

- **By making a reference null**

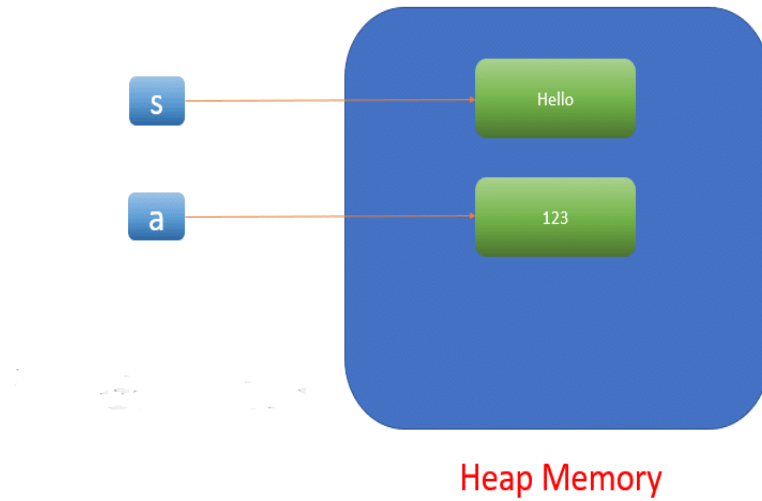
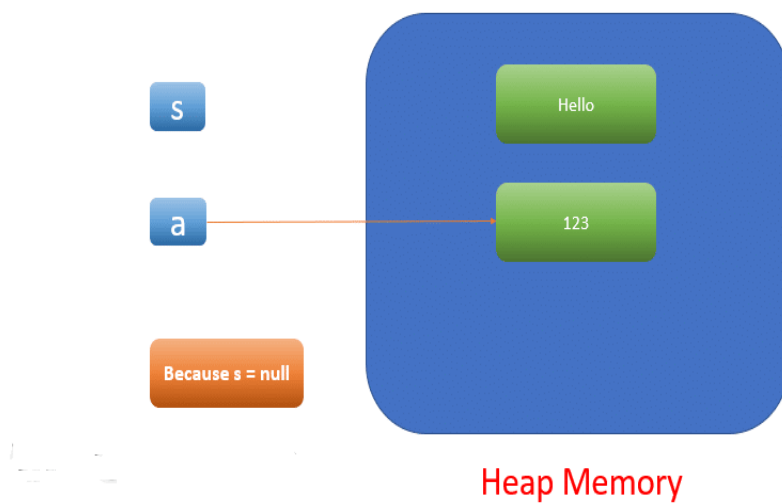
```
Student student = new Student();
student = null;
```

- **By assigning a reference to another**

```
Student studentOne = new Student();
Student studentTwo = new Student();
studentOne = studentTwo; // now the first object referred by studentOne is available for garbage collection
```

- **By using an anonymous object**

```
register(new Student());
```

**Example [Heap memory and referenced by reference variables]****Example [Heap memory and referenced by reference variables]**

## Enhancements to switch

- Traditional switch (Before Java 14) has following limitations:
  - Required break to prevent fall-through.
  - Verbose and error-prone.
  - Not an expression — could not return a value.
- What's New in Java 21?
- 1. **Arrow Syntax (->)** : Eliminating the need for break.
- 2. **Switch as an Expression** : You can now use switch to return values, making it an expression.
- 3. **Multiple Case Labels**: You can combine multiple values in a single case using **commas**:
- 4. **Block Body with yield** : When you need a block in a case (e.g., for multiple statements), use yield to return a value.
- 5. **Pattern Matching** : This allows matching types directly inside the case.

## Garbage Collection with enhancements to switch

- Enhanced switch features—including switch expressions, arrow syntax, and pattern matching—do not change how the Garbage Collector (GC) works, but they significantly influence object reachability and memory allocation patterns.
- **Scope and Reachability** : Enhanced switch features refine variable scoping, which directly impacts when objects become eligible for garbage collection.
  - **Block-Level Scoping**: In traditional switch statements, a variable declared in one case is often visible to subsequent cases due to fall-through, potentially keeping objects reachable longer than intended.
  - **Case-Specific Scoping**: Switch expressions and arrow syntax (->) often use specific code blocks {} for each case. Variables declared within these blocks go out of scope as soon as the block finishes, making their associated objects eligible for GC immediately.



## Garbage Collection with enhancements to switch

- **Pattern Matching and Temporary Objects** : Pattern Matching for switch (Java 21) introduces new ways to interact with the heap:
  - **Automatic Casting**: The JVM automatically creates "pattern variables" (e.g., `case String s`). These are local references on the stack that do not create new objects on the heap, thus adding no extra burden to the GC.
  - **Record Patterns**: When deconstructing Records (e.g., `case Point(int x, int y)`), the JVM extracts fields directly. This avoids creating temporary wrapper objects that would otherwise need to be collected in the Young Generation (Eden space).

## Garbage Collection effect

| Feature                      | Primary GC Impact                                                                          |
|------------------------------|--------------------------------------------------------------------------------------------|
| <b>Arrow Syntax (-&gt;)</b>  | Prevents accidental reachability caused by "fall-through" bugs.                            |
| <b>Switch Expressions</b>    | Encourages "single-assignment" patterns, making reference lifecycles clearer for GC roots. |
| <b>Pattern Matching</b>      | Reduces the need for temporary casting/wrapper objects on the heap.                        |
| <b>Record Deconstruction</b> | Enables direct field access, minimizing short-lived object allocation in Eden space.       |

## var in java

- The **var** identifier in Java, introduced in Java 10, enables local variable type inference. This means the compiler automatically determines the variable's type based on the value assigned to it, rather than requiring the developer to explicitly declare the type.
- **Type Inference:** The compiler infers the actual, static type of the variable at compile time from its initializer.
- **Static Typing Remains:** Java remains a statically-typed language. The var keyword is merely a syntactic shortcut; the inferred type is fixed and cannot be changed later at runtime.
- **Code Readability:** The primary purpose is to reduce boilerplate code and improve readability, especially with complex generic types (e.g., `var map = new HashMap<Integer, String>();` instead of the verbose explicit declaration).

## var in java : Usage Rules and Limitations

- **Local Variables Only:** var can only be used for local variables inside methods, in enhanced for-loops, traditional for-loops, and try-with-resources statements. It cannot be used for fields (instance or static variables), method parameters, or return types.
- **Must be Initialized:** A local variable declared with var must always be initialized on the same line so the compiler has a value from which to infer the type.
  - Correct: `var x = 5;`      Incorrect: `var x;`
- **Cannot be assigned null without a cast:** The compiler cannot infer a type from a null value alone.
- **Not a Keyword:** var is a reserved type name, not a keyword, which means existing code using var as a variable or method name will not break (though using it as a class name will cause issues)

## var in java : Examples

| Traditional Declaration  | var Declaration                                            | Inferred Type                        |
|-----------------------------------------------------------------------------------------------------------|------------------------------------------------------------|--------------------------------------|
| <code>int num = 100;</code>                                                                               | <code>var num = 100;</code>                                | <code>int</code>                     |
| <code>String name = "Java";</code>                                                                        | <code>var name = "Java";</code>                            | <code>String</code>                  |
| <code>List&lt;String&gt; list = new<br/>ArrayList&lt;&gt;();</code>                                       | <code>var list = new<br/>ArrayList&lt;String&gt;();</code> | <code>ArrayList&lt;String&gt;</code> |

## Unnamed Variable

- In Java 22, unnamed variables (and unnamed patterns) were officially finalized as a feature.
- They allow you to declare a variable that you do not intend to use by using an underscore (`_`) as its name. This improves code readability by signaling to other developers (and the compiler) that the value is irrelevant.
- Key Examples
- Catching Exceptions : When you must catch an exception but don't need to access the exception object itself.

```
String s = "abc";
try {
 int i = Integer.parseInt(s);
 // use i ...
} catch (NumberFormatException _) { // Unnamed variable for the exception
 System.out.println("Bad number: " + s);
}
```

## Unnamed Variable

- Enhanced For Loops : When iterating over a collection where only the number of iterations or the side effect of the loop body matters, the element variable can be unnamed..

```
int count = 0;
for (String _ : myList) { // Unnamed variable for the element
 count++;
}
```

- Lambda Expressions : In functional interfaces, some parameters might be irrelevant to the operation being performed.

```
import java.util.stream.Collectors;
// ...
stream.collect(Collectors.toMap(String::toUpperCase, _ -> "NODATA"));
// The lambda parameter '_' is not used to compute the value "NODATA"
```

## Unnamed Variable

- Try-with-resources : If the primary purpose of a resource is its automatic closing (a side effect) and the variable itself isn't used in the try block, it can be unnamed.

```
try (var _ = ScopedContext.acquire()) { // Unnamed resource variable
 // ... no use of the acquired context resource ...
}
```

- Record Patterns : When pattern matching with records, you can ignore specific components of a record using an unnamed pattern ( \_) to avoid declaring unnecessary variables.

```
record Point(int x, int y) {}
void printX(Object obj) {
 // Extracts 'x' but ignores 'y'
 if (obj instanceof Point(int x, _)) {
 System.out.println("X coordinate is: " + x);
 }
}
```

## Unnamed Variable

- **Note:** You cannot use an unnamed variable as a top-level variable that you intend to use later in the code; attempting to reference `_` will result in a compiler error. The underscore signals intentional non-use.

## Unnamed Variable and GC

- Unnamed variables, denoted by an underscore (`_`) (a preview feature in Java 21+), are placeholders for values that are present but not used, so they aren't bound to a name.
- **Eligibility for GC:** An object assigned to an unnamed variable becomes eligible for GC based purely on its reachability, just like any other object. Since an unnamed variable explicitly indicates the value is unused, the object can become unreachable and thus eligible for garbage collection almost immediately, or as soon as the expression creating it is finished, if no other reference exists.
- `new Person("John Doe"); // Unnamed object (no variable to refer to it)`
- The Person object created here is immediately eligible for garbage collection because no reference variable points to it.

## Unnamed class and GC

- Unnamed classes (a preview feature since Java 21) are a language construct used for simple, small programs, and they cannot be referenced by name.
- **Eligibility for GC:** Instances (objects) of unnamed classes are garbage collected according to the same reachability rules as instances of named classes. The GC process operates on objects in the heap, and their class definition's "unnamed" status does not change the fundamental mechanism.
- **Class Unloading:** The class definition itself (metadata) is stored in a different memory area (Metaspace since Java 8) and can also be garbage collected (unloaded) if the ClassLoader that loaded it is no longer reachable, and no instances of the class are in use.

## Unnamed class, variable and GC

- **Note :** The naming of classes or variables has no special impact on Java's automatic garbage collection system.
- GC focuses exclusively on the reachability of objects from a set of "GC roots" (e.g., active threads, static variables, local variables on the stack). If an object is not reachable, it is eligible for collection.

## String template and GC

- String templates in Java, introduced as a preview feature in Java 21 and finalized in Java 22, are subject to the same automatic garbage collection rules as any other Java object.
- The string template feature was removed in JDK 23 to be rethought. As such, it is not part of the standard Java language at present.
- The primary goal of string templates is to provide a more readable and convenient way to create strings, not to change the underlying memory management (GC) behavior.
- Now focus on **String Concatenation**, **StringBuilder**, **String.format()** / **String.formatted()**.

